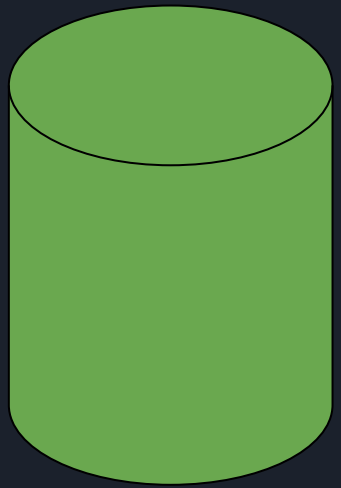
GESTIÓN DE DATOS

PL/SQL - FUNCIONES - CICLOS

RECORDERIS...



¿QUÉ APRENDEREMOS HOY?



- Manejo de ciclos en las funciones
- Diferentes formas de definir ciclos en mysql y MariaDB

Recorderis: FUNCIÓN



Recorderis: PL/SQL



Recorderis: Función pl/sql



Pregunta: ¿Cuando usar ciclos?



¿Cuando FOR y cuando WHILE?

(Discusión verbal)

¿Cuando FOR y cuando WHILE?

Aunque es una decisión muy personal, una guía es esta:

Usar FOR cuando se conozca la cantidad de iteraciones

Usar WHILE cuando no se conozcan la cantidad de iteraciones o cuando haya incertidumbre

Ciclos usando WHILE

```
(...)
create function fn_my_first_while(
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result integer default 1;
   declare v_current integer default 0;

   while   v_current < p_loops  do
       set v_result  := v_result + v_result;
       set v_current := v_current + 1;
   end while;

   return v_result;
end;
(...)
```

Ciclos usando WHILE

Debe existir una condición para salir del repeat. Sino, habrá ciclo infinito!

Todo while ***debe*** cerrarse con end while

```
(...)
create function fn_my_first_while(
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result integer default 1;
   declare v_current integer default 0;

   while v_current < p_loops do
      set v_result  := v_result + v_result;
      set v_current := v_current + 1;
   end while;

   return v_result;
end;
(...)
```

Ciclos usando REPEAT

```
(...)
create function fn_my_first_repeat (
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result integer default 1;
   declare v_current  integer default 0;

   repeat
       set v_result  := v_result + v_result;
       set v_current := v_current + 1;
   until v_current  = p_loops
   end repeat;

   return v_result;
end;
(...)
```

Ciclos usando REPEAT

Debe existir una condición para salir del repeat. Sino, habrá ciclo infinito!

Todo repeat ***debe*** cerrarse con end repeat

```
(...)
create function fn_my_first_repeat (
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result integer default 1;
   declare v_current  integer default 0;

   repeat
      set v_result  := v_result + v_result;
      set v_current := v_current + 1;
   until v_current  = p_loops
   end repeat;

   return v_result;
end;
(...)
```

Ciclos usando LOOP

```
(...)
create function fn_my_first_loop(
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result  integer default 1;
   declare v_current integer default 1;

   my_loop:  loop
       set v_result  := v_result * v_current;
       set v_current := v_current + 1;

       if v_current = p_loops then
           leave my_loop;
       end if;
   end loop;

   return v_result;
end;
(...)
```

Ciclos usando LOOP

Cada loop debe tener obligatoriamente un nombre

Debe existir una condición para salir del loop usando la palabra clave "leave", indicando el nombre del loop

Todo loop ***debe*** cerrarse con end loop

```
(...)
create function fn_my_first_loop(
   p_loops integer
)
returns integer
deterministic
begin
   declare v_result  integer default 1;
   declare v_current integer default 1;

   my_loop: loop
      set v_result  := v_result * v_current;
      set v_current := v_current + 1;

      if v_current = p_loops then
         leave my_loop;
      end if;
   end loop;

   return v_result;
end;
(...)
```

Ciclos usando FOR










```
(...)
create function fn_my_first_for(
   p_lower_limit int,
   p_upper_limit int
)
returns int
begin
   declare v_result integer default 0;

   for i in p_lower_limit..p_upper_limit do
       set v_result := v_result + i;
   end for;

   return v_result;
end;
(...)
```

Ciclos usando FOR

Se especifica valor inicial y valor final

Todo for ***debe*** cerrarse con end for

```
(...)
create function fn_my_first_for(
   p_lower_limit int,
   p_upper_limit int
)
returns int
begin
   declare v_result integer default 0;

   for i in p_lower_limit..p_upper_limit do
      set v_result := v_result + i;
   end for;

   return v_result;
end;
(...)
```

RESUMEN HASTA AHORA

while - do - end while	Manejo de ciclos especificando la condición después de "while" y antes de "do"; ***útil para cuando no se conozca la cantidad de iteraciones***
for - end for	Es la más similar a otros lenguajes de programación. ***Útil para cuando se conozca la cantidad de iteraciones*** evitando un manejo manual. ***Funciona en MariaDB pero no en MySQL***
repeat - until - end repeat	Manejo de ciclos, ***garantizando que se ejecuta al menos una vez***, hasta que se cumpla la condición en "until"
loop - end loop (en desuso; conocerla por compatibilidad)	Manejo de ciclos especificando un nombre a cada loop (y así poder identificar ciclos anidados). Se sale del ciclo con la palabra clave "leave", seguida del nombre del loop.

Consejo: No usar ciclos cuando basta con un simple SELECT

EJERCICIOS (1/4)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_summation_01
- Parámetros: valor inferior, valor_superior (p_min, p_max)
- Tipo de dato de retorno: int
- Descripción: Realizar la sumatoria incluyendo valor_inferior y valor_superior
- Ejemplo: Para los valores de entrada 1 y 10 el cálculo sería 1+2+3+4+5+6+7+8+9+10 = 55

EJERCICIOS (2 / 4)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_summation_02
- Parámetros: valor inferior, valor_superior (p_min, p_max)
- Tipo de dato de retorno: int
- Descripción: Realizar la sumatoria incluyendo valor_inferior y valor_superior pero sólo para números impares (es decir, debe apoyarse en la función fn_number_type_01 creada en la clase anterior)
- Ejemplo: Para los valores de entrada 1 y 10 el cálculo sería 1+3+5+7+9 = 25

EJERCICIOS (3 / 4)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_factorial_01
- Parámetros: valor 1 (p_val1)
- Tipo de dato de retorno: int
- Descripción: Calcular el factorial para el número especificado en p_val1. Si el número de entrada es 0 ó negativo entonces el valor a retornar es 0.
- Ejemplo: Para el valor 5 el resultado es 5! = 1*2*3*4*5 = 120.

EJERCICIOS (4 / 4)

Escribir una función en pl/sql con las siguientes características:

- Nombre: fn_factorial_summation_01
- Parámetros: valor 1 (p_val1)
- Tipo de dato de retorno: int
- Descripción: Calcular la suma de dígitos del factorial para el número especificado en p_val1. Es decir, la suma de dígitos de la función en el ejercicio anterior
- Ejemplo: Para el valor 5 el factorial es 120, el resultado es 1+2+0 = 3.

¿PREGUNTAS?

